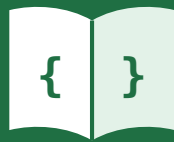

Module 21 — gRPC

Lesson Notes

Spring Boot Edu



2026-09-25

Contents

1. Learning Goals	2
2. Concepts	2
2.1 What gRPC Is	2
2.2 REST and gRPC Compared	2
2.3 Four Kinds of RPC	3
3. Step-by-Step Examples	3
3.1 The Contract and the Code Generation	4
3.2 A Unary Service	5
3.3 Streaming	6
3.4 Errors: Status Codes	8
3.5 Interceptors	8
3.6 Clients and Deadlines	9
3.7 Health, Reflection and Tests	11
4. Common Mistakes and Best Practices	12
5. Summary	12
6. Further Reading	12

1. Learning Goals

By the end of this module you will be able to:

- Write a service contract in Protocol Buffers and generate Java code from it with Maven
- Implement unary, server streaming, client streaming and bidirectional streaming RPCs with `@GrpcService`
- Map Java exceptions to gRPC status codes with `@GrpcAdvice`
- Add a server interceptor, and call services through client stubs with deadlines
- Use the standard health and reflection services, and test everything in-process
- Decide between REST and gRPC for a given API

Prerequisites: Module 03 (REST APIs) · **Estimated time:** 4 hours · **No Docker needed**

2. Concepts

2.1 What gRPC Is

gRPC is a remote procedure call framework. The client calls a method on a generated **stub** as if it were a local object, and the call goes over HTTP/2 to the server. The contract is a `.proto` file. Messages are encoded as **Protocol Buffers**: a compact binary format in which each field is identified by its **number**, not by its name.

2.2 REST and gRPC Compared

	REST (JSON over HTTP)	gRPC (Protobuf over HTTP/2)
Contract	optional (OpenAPI), often written after the code	required, the <code>.proto</code> file comes first
Payload	JSON text, readable	binary, small and fast to parse
Code	hand-written client or generated from OpenAPI	client and server stubs generated for many languages
Streaming	limited (SSE, WebSocket)	built in: client, server and bidirectional
Errors	HTTP status + body (ProblemDetail)	16 status codes + description + metadata
Browsers	work directly	need a proxy (gRPC-Web)

	REST (JSON over HTTP)	gRPC (Protobuf over HTTP/2)
Tools	curl, browser	grpcurl, Postman, reflection
Good for	public APIs, web clients, caching	internal service-to-service calls, streaming, many languages

In the capstone, the gateway offers REST to the outside, and the internal services talk gRPC.

2.3 Four Kinds of RPC

Kind	Request	Response	Example in this lesson
Unary	one	one	GetBook
Server streaming	one	many	ListBooks
Client streaming	many	one	ImportBooks
Bidirectional streaming	many	many, at any time	Chat (stock questions and answers)

3. Step-by-Step Examples

Start the application. The gRPC server listens on port 9090, and the tour calls it through a client stub:

```
./mvnw -pl modules/21-grpc/lesson spring-boot:run
```

```
=== 3.2 Unary: GetBook ===
Effective Java by Joshua Bloch, 8990 cents, 48 bytes on the wire
=== 3.3 Server streaming: ListBooks ===
Effective Java
Java Puzzlers
...
=== 3.4 Errors are status codes ===
NOT_FOUND: No book with ISBN 00000000000000
```

The same book as compact JSON is 91 bytes: the field names travel with every message. With reflection enabled, [grpcurl](#) can call the server without the .proto file (see the README).

i Note

On JDK 27, protobuf-java prints a warning about `sun.misc.Unsafe`. It is harmless for now; the JVM option `--sun-misc-unsafe-memory-access=allow` hides it until protobuf-java moves to the new memory APIs.

3.1 The Contract and the Code Generation

```
syntax = "proto3";

package bookstore.catalog.v1;

option java_package = "com.springbootedu.grpc.catalog.v1";
option java_multiple_files = true;

service BookCatalog {
    rpc GetBook (GetBookRequest) returns (Book);           // unary
    rpc ListBooks (ListBooksRequest) returns (stream Book); // server streaming
    rpc ImportBooks (stream Book) returns (ImportSummary); // client streaming
    rpc Chat (stream StockQuestion) returns (stream StockAnswer); // bidirectional
    ↪ streaming
}

message GetBookRequest {
    string isbn = 1;
}

message ListBooksRequest {
    string author = 1;           // empty: all books
}

message Book {
    string isbn = 1;
    string title = 2;
    string author = 3;
    int64 price_cents = 4;       // money as whole cents: no floating point errors
}

message ImportSummary {
    int32 imported = 1;
    int32 rejected = 2;
}
```

```
message StockQuestion {
    string isbn = 1;
}

message StockAnswer {
    string isbn = 1;
    int32 available = 2;
}
```

- package is the gRPC name (bookstore.catalog.v1.BookCatalog/GetBook), java_package the Java package of the generated code.
- stream before a type makes that side a stream.
- Money is an integer of cents: double would bring rounding errors.

Spring Boot manages the protobuf-maven-plugin of ascpes and configures it through spring-boot-starter-parent. It reads src/main/proto and writes the messages and the BookCatalogGrpc class to target/generated-sources/protobuf:

```
<!-- generates Java messages and gRPC stubs from src/main/proto (configured by
↳ spring-boot-starter-parent) -->
<plugin>
  <groupId>io.github.ascpes</groupId>
  <artifactId>protobuf-maven-plugin</artifactId>
</plugin>
```

3.2 A Unary Service

A service extends the generated ...ImplBase class. @GrpcService makes it a bean that the gRPC server exposes:

```
@Override
public void getBook(GetBookRequest request, StreamObserver<Book> responses) {
    if (!request.getIsbn().matches("\\d{13}")) {
        throw new IllegalArgumentException("An ISBN has 13 digits: " +
            ↳ request.getIsbn()); // → INVALID_ARGUMENT
    }
    Book book = store.find(request.getIsbn()).orElseThrow(() -> new
        ↳ BookNotFoundException(request.getIsbn()));
    responses.onNext(book); // exactly one answer ...
    responses.onCompleted(); // ... and the call is over
}
```

The answer goes to a StreamObserver: onNext sends a message, onCompleted ends the call successfully, onError ends it with a status.

3.3 Streaming

Server streaming: the server calls onNext as often as it has results:

```
@Override
public void listBooks(ListBooksRequest request, StreamObserver<Book> responses) {
    store.byAuthor(request.getAuthor()).forEach(responses::onNext);    // many answers,
    ↪ one after the other
    responses.onCompleted();
}
```

Client streaming: the method returns an observer that receives the client's messages. The answer comes when the client calls onCompleted:

```
@Override
public StreamObserver<Book> importBooks(StreamObserver<ImportSummary> responses) {
    return new StreamObserver<>() {                                // the server returns an observer
        ↪ for the client's stream
        private int imported;
        private int rejected;

        @Override
        public void onNext(Book book) {
            if (book.getIsbn().matches("\\d{13}")) {
                store.add(book, 1);
                imported++;
            } else {
                rejected++;
            }
        }

        @Override
        public void onError(Throwable error) {
            // the client cancelled or the connection broke: nothing to answer
        }

        @Override
        public void onCompleted() {                                // the client has sent everything:
            ↪ one summary
            re-
            ↪ sponses.onNext(ImportSummary.newBuilder().setImported(imported).setRejected(rejected).build());
            responses.onCompleted();
        }
    };
}
```

Bidirectional streaming: both sides send whenever they want, and the stream stays

open until both are finished:

```
@Override
public StreamObserver<StockQuestion> chat(StreamObserver<StockAnswer> responses) {
    return new StreamObserver<>() {
        @Override
        public void onNext(StockQuestion question) {    // answer each question at once,
            ↪ the stream stays open
            responses.onNext(StockAnswer.newBuilder()
                .setIsbn(question.getIsbn())
                .setAvailable(store.stockOf(question.getIsbn()))
                .build());
        }

        @Override
        public void onError(Throwable error) {
        }

        @Override
        public void onCompleted() {
            responses.onCompleted();
        }
    };
}
```

On the client side, streams need the asynchronous stub (BookCatalogStub). The test sends three books into the import stream and receives one summary:

```
@Test
void clientStreaming() throws Exception {
    var summary = new CompletableFuture<ImportSummary>();
    StreamObserver<Book> upload = asyncCatalog.importBooks(new StreamObserver<>() {
        @Override
        public void onNext(ImportSummary value) {
            summary.complete(value);
        }

        @Override
        public void onError(Throwable error) {
            summary.completeExceptionally(error);
        }

        @Override
        public void onCompleted() {
        }
    });
}
```

```

upload.onNext(book("9781098150358", "Learning Spring Boot 3.0"));
upload.onNext(book("123", "Broken ISBN"));
upload.onNext(book("9781617294945", "Spring Microservices in Action"));
upload.onCompleted(); // the client says: no more books

ImportSummary result = summary.get(5, TimeUnit.SECONDS);
assertThat(result.getImported()).isEqualTo(2);
assertThat(result.getRejected()).isEqualTo(1);
}

```

3.4 Errors: Status Codes

gRPC has its own status codes: OK, NOT_FOUND, INVALID_ARGUMENT, DEADLINE_EXCEEDED, UNAVAILABLE, PERMISSION_DENIED, ... An exception without a mapping reaches the client as UNKNOWN, without a useful message. @GrpcAdvice works like @ControllerAdvice:

```

@GrpcAdvice
public class CatalogExceptionAdvice { // public: Spring gRPC calls the
    ↪ handlers reflectively

    @GrpcExceptionHandler
    public Status notFound(BookNotFoundException e) {
        return Status.NOT_FOUND.withDescription(e.getMessage());
    }

    @GrpcExceptionHandler
    public Status invalid(IllegalArgumentException e) {
        return Status.INVALID_ARGUMENT.withDescription(e.getMessage());
    }
}

```

! Warning

The advice class and its handler methods must be public. Spring gRPC calls them reflectively. A package-private handler fails with `IllegalAccessException`, and the client receives UNKNOWN.

The blocking stub throws a `StatusRuntimeException`. Its `getStatus()` contains the code and the description.

3.5 Interceptors

A server interceptor wraps every call, like a servlet filter. This one logs the method, the final status and the time:

```

@Component
@GlobalServerInterceptor // applies to all gRPC services
↪ of this server
@Order(Ordered.HIGHEST_PRECEDENCE) // outermost: it also sees
↪ calls closed by the error mapping
public class CallLoggingInterceptor implements ServerInterceptor {

    private static final Logger log =
        ↪ LoggerFactory.getLogger(CallLoggingInterceptor.class);

    private final List<String> recentCalls = new CopyOnWriteArrayList<>();

    @Override
    public <Q, R> ServerCall.Listener<Q> interceptCall(ServerCall<Q, R> call, Metadata
        ↪ headers,
                                                    ServerCallHandler<Q, R> next) {

        long start = System.nanoTime();
        String method = call.getMethodDescriptor().getFullMethodName();
        ServerCall<Q, R> timedCall = new SimpleForwardingServerCall<>(call) {
            @Override
            public void close(Status status, Metadata trailers) { // called once, when
                ↪ the call ends
                    log.info("{} {} in {} ms", method, status.getStatusCode(), (System.nanoTime() -
↪ start) / 1_000_000);
                    recentCalls.add(method + " " + status.getStatusCode());
                    super.close(status, trailers);
                }
            };
            return next.startCall(timedCall, headers);
        }
    }
}

```

@GlobalServerInterceptor applies it to all services. @Order(HIGHEST_PRECEDENCE) makes it the outermost interceptor, so it also sees the calls that the error mapping closes. Typical interceptors check authentication, add trace IDs or measure metrics.

3.6 Clients and Deadlines

@ImportGrpcClients creates stub beans for a **logical channel**. The real address is configuration:

```

@Configuration(proxyBeanMethods = false)
@ImportGrpcClients(target = "catalog", types = {
    BookCatalogGrpc.BookCatalogBlockingStub.class,
    ↪ BookCatalogGrpc.BookCatalogStub.class})
class ClientConfiguration {

```

```

}

grpc:
  server:
    port: 9090 # the default; gRPC runs on its own Netty
    ↪ server (HTTP/2)
  client:
    channel:
      catalog: # the logical name used by @ImportGrpcClients
        target: static://localhost:9090 # here the application calls itself (see
    ↪ LessonTour)
      default:
        deadline: 2s # every call without its own deadline
    ↪ gives up after 2 s

```

A **deadline** is the point in time after which the client gives up. It travels with the call to the server, which can stop working on it. Without a deadline, a call to a hanging server can wait forever:

```

@Component
public class CatalogClient {

    private final BookCatalogGrpc.BookCatalogBlockingStub catalog;

    CatalogClient(BookCatalogGrpc.BookCatalogBlockingStub catalog) { // imported by
    ↪ @ImportGrpcClients
        this.catalog = catalog;
    }

    public String titleOf(String isbn, Duration deadline) {
        return catalog.withDeadlineAfter(deadline.toMillis(), TimeUnit.MILLISECONDS) //
        ↪ give up after this time
            .getBook(GetBookRequest.newBuilder().setIsbn(isbn).build())
            .getTitle();
    }
}

```

```

@Test
void aTooShortDeadlineEndsTheCall() {
    assertThatThrownBy(() -> client.titleOf("9780134685991", Duration.ofMillis(100)))
    ↪ // server needs 300 ms
        .isInstanceOfSatisfying(StatusRuntimeException.class,
            e ->
    ↪ assertThat(e.getStatus().getCode()).isEqualTo(Status.Code.DEADLINE_EXCEEDED));
}

```

★ Tip

Set a default deadline for every channel (`default.deadline`), and a shorter one for calls on a user's request path. A deadline is absolute: a service that calls further services passes the rest of the time on.

3.7 Health, Reflection and Tests

With `io.grpc:grpc-services` on the classpath, Boot registers two standard services:

- **Health** (`grpc.health.v1.Health`): Boot's health indicators decide whether the server reports `SERVING`. Kubernetes can use it as a gRPC probe (module 22).
- **Reflection**: clients like `grpcurl` can list the services and download their descriptions.

`@AutoConfigureTestGrpcTransport` replaces every channel and the server with **in-process** transport. The tests need no network port and are fast, yet they go through the real stubs, the serialization and the interceptors:

```
@SpringBootTest(properties = "bookstore.tour.enabled=false")
@AutoConfigureTestGrpcTransport // in-process channels: fast, no
↪ network port
class BookCatalogServiceTest {

    @Autowired
    BookCatalogGrpc.BookCatalogBlockingStub catalog; // the application's stub
    ↪ (ClientConfiguration), now in-process

    @Autowired
    BookCatalogGrpc.BookCatalogStub asyncCatalog; // for client and bidirectional
    ↪ streaming

    @Test
    void unaryCall() {
        Book book = cata-
        ↪ log.getLogBook(GetBookRequest.newBuilder().setIsbn("9780134685991").build());

        assertThat(book.getTitle()).isEqualTo("Effective Java");
        assertThat(book.getPriceCents()).isEqualTo(8990);
    }
}
```

4. Common Mistakes and Best Practices

!! Caution

Never change the number or the type of an existing field, and never reuse the number of a deleted field (mark it reserved). The number is the only thing on the wire: old clients would read wrong data without any error.

- **Do:** add new fields with new numbers. Old clients ignore unknown fields, and new code sees default values (0, "") in old messages.
- **Don't:** let exceptions reach clients as UNKNOWN. Map them to status codes that the client can act on.
- **Do:** set a deadline on every call, and handle DEADLINE_EXCEEDED and UNAVAILABLE (retry or fallback).
- **Don't:** block the stream observer of a bidirectional stream for a long time. Answer or hand the work to another thread.
- **Do:** version the package (bookstore.catalog.v1). A breaking change becomes v2, and both run side by side for a while.

5. Summary

- The .proto file is the contract. The Maven plugin generates messages and stubs, and field numbers define the wire format.
- @GrpcService beans implement unary and streaming methods with StreamObserver.
- @GrpcAdvice maps exceptions to status codes, and @GlobalServerInterceptor adds cross-cutting behaviour.
- @ImportGrpcClients creates stubs for logical channels. Deadlines protect callers from slow servers.
- Health and reflection come with grpc-services, and @AutoConfigureTestGrpcTransport tests everything in-process.
- REST fits public and browser APIs, gRPC fits internal, typed, streaming service-to-service calls.

Next step: [Exercises](#)

6. Further Reading

- [Spring Boot — gRPC](#)
- [Spring gRPC Reference](#)
- [gRPC — Core concepts](#) · [Status codes](#) · [Deadlines](#)
- [Protocol Buffers — Language Guide \(proto3\)](#) · [Updating a message type](#)